資料結構 — HW3

作者：蔡品辰

學號：41243154

December 31, 2024.

目錄

Homework_3 3
解題說明 3
題目問題 4
定義及過程 5
程式碼實作 6
結構 Node 6
物件 Polynomial 6
建構子(constructor) 7
複製建構子(copy constructor) 7
解構子(destructor) 8
運算子多載(=) 8
運算子多載(+) 9
運算子多載(-) 10
運算子多載(*) 11
多項式求值(Evaluate) 12
新增(AddTerm) 13
清除(Clear) 14
輸入多載(>>) 14
輸出多載(<<) 15
主程式(main) 16
效能分析(Analysis) 18
1. 加法(operator+) 18
2. 減法(operator -) 18
3. 乘法(operator*) 18
4. 多項式求值(Evaluate) 19

5. 整體(total) 19
執行結果(Testing) 19
驗證計算(Proving) 20
效能量測(Measuring) 20
申論及開發報告 21

Homework_3

- 解題說明

[Programmming Project]

Develop a C++ calss Polynomial to represent and manipulate polynomials with integer coefficients (use circular linked lists with header nodes). Each term of the polynomial will be represented as a node. Thus a node in this system will have three data members as below:

coef	exp	link

Each polynomial is to be represented as a list with header node. To delete polynomials efficiently, we need to use an available-space list and associated functions as described in Section 4.5. The external (i.e., for input or output) representation of a univariate polynomial will be assumed to be a sequence of integers of the form: n, c1, e1, e2, c3, e3, ..., cn, en.

where ei represents an exponent and ci a coefficient; n gives the number of terms in the polynomial.

The exponents are in decreasing order e1> e2 > ⋯ > en.

● 題目問題

Write and test the following functions:

a. istream& operator>>(istream& is, Polynomial& x): Read in an input polynomial and convert it to its circular list representation using a header node.
b. ostream& operator<<(ostream& os, Polynomial& x): Convert x from its linked list representation to its external representation and output it.
c. Polynomial :: Polynomial(const Polynomial& a) [Copy Constructor]: Initialize the polynomial *this to the polynomial a.
d. const Polynomial& Polynomial::operator=(const Polynomial& a) const [Assignment Operator]: Assign polynomial a to *this.
e. Polynomial:: ~Polynomial() [Destructor]: Return all nodes of the polynomial *this to the available-space list.
f. Polynomial operator+ (const Polynomial& b) const [Addition]: Create and return the polynomial *this + b.
g. Polynomial operator- (const Polynomial& b) const [Subtraction]: Create and return the polynomial *this - b.
h. Polynomial operator*(const Polynomial& b) const [Multiplication]: Create and return the polynomial *this * b.
i. float Polynomial::Evaluate (float x) const: Evaluate the polynomial *this at x and return the result.

● 定義及過程

開發一個 C++類別 `Polynomial`，用來表示和操作具有整數係數的多項式 (使用帶有標頭節點的**循環連結串列**)。

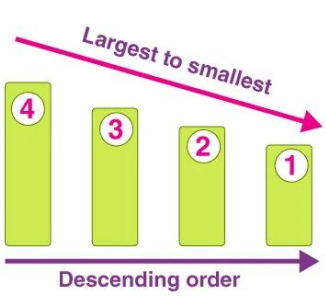
多項式的每一項將表示為一個節點(node)，每個節點(node)包含以下三個資料成員：

✧ `coef`：整數係數(integer coefficients)。
✧ `exp` ：指數。
✧ `link`：指向下一個節點的指標。

每一個多項式將使用帶有**標頭節點**的串列表示。 為了高效刪除多項式，我們需要使用一個可以使用空間串列及相關函數，如 4.5 節所述。

多項式的外部表示(例如輸入或輸出) 將假設為以下形式的整數序列：`n, c1, e1, c2, e2, c3, e3, …, cn, en`，其中：

✧ `n` 是多項式中項目的數量。
✧ `ci` 是第 i 項的係數。
✧ `ei` 是第 i 項的指數，以降幕的方式排列。



1. 加法(operator+)：回傳二多項式相加，並化簡之結果。
2. 減法(operator -)：回傳二多項式相減，並化簡之結果。
3. 乘法(operator*)：回傳二多項式相乘，並化簡之結果。
4. 求值(Evaluate)：輸入未知數，並求多項式之值。

- 程式碼實作

  - 結構 **Node**

```
struct Node{

    int coef;   //integer coefficients

    int exp;    //exponent

    Node* link; //point to next node

};
```

  - 物件 Polynomial

```
class Polynomial{

public:

    Polynomial();                 //constructor

    Polynomial(const Polynomial& a); //copy constructor

    ~Polynomial();                //destructor

    const Polynomial& operator=(const Polynomial& a); //assignment operator

    Polynomial operator+(const Polynomial& b) const; //addition operator

    Polynomial operator-(const Polynomial& b) const; //subtraction operator

    Polynomial operator*(const Polynomial& b) const; //multiplication operator

    float Evaluate(float x) const;



    friend istream& operator>>(istream& is, Polynomial& x);

    friend ostream& operator<<(ostream& os, Polynomial& x);



private:
```

```
    Node* header;                  //header node

    void AddTerm(int coef, int exp); //insert new term

    void Clear();                  //clear polynomial

};
```

- 建構子(constructor)

```
Polynomial::Polynomial(){

    header = new Node{0, 0, NULL};

    header->link = header;

}
```

- 複製建構子(copy constructor)

```
Polynomial::Polynomial(const Polynomial& a){

    header = new Node{0, 0, NULL};

    header->link = header;

    Node* check = a.header->link;

    while(check != a.header){

        AddTerm(check->coef, check->exp);

        check = check->link;

    }

}
```

- 解構子(destructor)

```
Polynomial::~Polynomial(){

    Clear();

    delete header;

}
```

- 運算子多載(=)

```
const Polynomial& Polynomial::operator=(const Polynomial& a){

    if(this == &a)return *this;

    Clear();

    Node* check = a.header->link;

    while(check != a.header){

        AddTerm(check->coef, check->exp);

        check = check->link;

    }

    return *this;

}
```

- 運算子多載(+)

```
Polynomial Polynomial::operator+(const Polynomial& b) const{

    Polynomial result;

    Node* p1 = this->header->link;

    Node* p2 = b.header->link;



    while(p1 != this->header || p2 != b.header){

        if(p1 == this->header || (p2 != b.header && p2->exp > p1->exp)){

            result.AddTerm(p2->coef, p2->exp);

            p2 = p2->link;

        }else if((p1 != this->header && p1->exp > p2->exp) || p2 == b.header){

            result.AddTerm(p1->coef, p1->exp);

            p1 = p1->link;

        }else{

            int newCoef = p1->coef + p2->coef;

            if(newCoef != 0)result.AddTerm(newCoef, p1->exp);

            p1 = p1->link;

            p2 = p2->link;

        }

    }

    return result;

}
```

- 運算子多載(-)

```
Polynomial Polynomial::operator-(const Polynomial& b) const{

    Polynomial result;

    Node* p1 = this->header->link;

    Node* p2 = b.header->link;



    while(p1 != this->header || p2 != b.header){

        if(p1 == this->header || (p2 != b.header && p2->exp > p1->exp)){

            result.AddTerm(p2->coef, p2->exp);

            p2 = p2->link;

        }else if((p1 != this->header && p1->exp > p2->exp) || p2 == b.header){

            result.AddTerm(p1->coef, p1->exp);

            p1 = p1->link;

        }else{

            int newCoef = p1->coef - p2->coef;

            if(newCoef != 0)result.AddTerm(newCoef, p1->exp);

            p1 = p1->link;

            p2 = p2->link;

        }

    }

    return result;

}
```

- **運算子多載(*)**

```
Polynomial Polynomial::operator*(const Polynomial& b) const{

    Polynomial result;

    Node* p1 = this->header->link;



    while(p1 != this->header){

        Node* p2 = b.header->link;

        while(p2 != b.header){

            int newCoef = p1->coef * p2->coef;

            int newExp = p1->exp + p2->exp;

            result.AddTerm(newCoef, newExp);

            p2 = p2->link;

        }

        p1 = p1->link;

    }

    return result;

}
```

- 多項式求值(Evaluate)

```
float Polynomial::Evaluate(float x) const{

    float result = 0;

    Node* check = header->link;

    while(check != header){

        result += check->coef * pow(x, check->exp);

        check = check->link;

    }

    return result;

}
```

- 新增(AddTerm)

```
void Polynomial::AddTerm(int coef, int exp){

    if(coef == 0)return;



    Node* check = header;

    while(check->link != header && check->link->exp > exp){

        check = check->link;

    }

    if(check->link != header && check->link->exp == exp){

        check->link->coef += coef;

        if(check->link->coef == 0){

            Node* temp = check->link;

            check->link = temp->link;

            delete temp;

        }

    }else{

        Node* newNode = new Node{coef, exp, check->link};

        check->link = newNode;

    }

}
```

- 清除(Clear)

```
void Polynomial::Clear(){

    Node* check = header->link;

    while(check != header){

        Node* temp = check; //save current node.

        check = check->link;//move to next node.

        delete temp; //delete current node.

    }

    header->link = header;

}
```

- 輸入多載(>>)

```
istream& operator>>(istream& is, Polynomial& x){

    x.Clear();

    int n;

    cout<<" Enter number of terms: ";

    is >> n;

    for(int i=0;i<n;i++){

        int coef,exp;

        cout<<"  Enter coefficient and exponent: "<<i+1<<": ";

        is >> coef >> exp;

        x.AddTerm(coef,exp);

    }

    return is;

}
```

- **輸出多載(<<)**

```
ostream& operator<<(ostream& os, Polynomial& x){

    Node* check = x.header->link;

    if(check == x.header){

        os<<"0";

        return os;

    }

    bool first = true;

    while(check != x.header){

        if(!first && check->coef > 0)os<<" + ";

        if(!first && check->coef < 0)os<<" - ";

        if(check->coef != 1 || check->exp == 0)os<<abs(check->coef);

        if(check->exp != 0){

            os<<"x";

            if(check->exp != 1)os<<"^"<<check->exp;

        }

        check = check->link;

        first = false;

    }

    return os;

}
```

- 主程式(main)

```
int main(){

    Polynomial p1, p2;

    cout << "Enter the first polynomial:" << endl;

    cin >> p1;

    cout << "Enter the second polynomial:" << endl;

    cin >> p2;



    cout << "First polynomial: " << p1 << endl;

    cout << "Second polynomial: " << p2 << endl;



    Polynomial sum = p1 + p2;

    Polynomial diff = p1 - p2;

    Polynomial prod = p1 * p2;



     cout<<endl;

    cout << "Sum: " << sum << endl;

    cout << "Difference: " << diff << endl;

    cout << "Product: " << prod << endl;



    float x;

     cout<<endl;

    cout << "Enter a value to evaluate the first polynomial: ";

    cin >> x;
```

```
    cout << "Value of the first polynomial at " << x << ": " << p1.Evaluate(x) << endl;



    return 0;

}
```

● 效能分析(Analysis)

1. 加法(operator+)
   - Time complexity
     - ✧ p1 和 p2 分別有 m 和 n 項。
     - ✧ 遍歷兩個多項式之節點需：O(m+n)。
     - ✧ 使用 AddTerm 合併結果，最差的情況為 O(m+n)。
     - ✧ 時間複雜度為 $O(m+n)^2$。
   - Space complexity
     - ✧ 空間複雜度為 O(m+n)。
2. 減法(operator -)
   - Time complexity
     - ✧ p1 和 p2 分別有 m 和 n 項。
     - ✧ 遍歷兩個多項式之節點需：O(m+n)。
     - ✧ 使用 AddTerm 合併結果，最差的情況為 O(m+n)。
     - ✧ 時間複雜度為 $O(m+n)^2$。
   - Space complexity
     - ✧ 空間複雜度為 O(m+n)。
3. 乘法(operator*)
   - Time complexity
     - ✧ p1 和 p2 分別有 m 和 n 項。
     - ✧ 遍歷兩個多項式之節點需：O(m+n)。
     - ✧ 使用 AddTerm 合併結果，最差的情況為 O(m+n)。
     - ✧ 時間複雜度為 $O(m+n)^2$。
   - Space complexity
     - ✧ 空間複雜度為 O(m*n)。

4. 多項式求值(Evaluate)
   - Time complexity
     - ✧ 遍歷 m 個節點。
     - ✧ 假設多項式中最大的指數為 exp，則 $O(\log_2(exp))$。
     - ✧ 時間複雜度為 $O(m * \log_2(exp))$。
   - Space complexity
     - ✧ 時間複雜度為 O(1)。
5. 整體(total)
   - Time complexity
     - ✧ 由乘法的 $O(m+n)^2$主導，設二個多項式的項數最大為 m 則：

       $O(m+m)^2 = O(m)^4$。
   - Space complexity
     - ✧ 由乘法的 O(m*n)主導，設二個多項式的項數最大為 m 則：

       $O(m*m) = O(m)^2$。

● 執行結果(Testing)

```
C:\Users\ping9\OneDrive\文件\.aino\虎科\2-1_資料結構\Homework_3\src>Hw3.exe
Enter the first polynomial:
 Enter number of terms: 2
  Enter coefficient and exponent: 1: 3 2
  Enter coefficient and exponent: 2: 5 0
Enter the second polynomial:
 Enter number of terms: 3
  Enter coefficient and exponent: 1: 1 3
  Enter coefficient and exponent: 2: 2 2
  Enter coefficient and exponent: 3: -4 0
First polynomial: 3x^2 + 5
Second polynomial: x^3 + 2x^2 - 4

Sum: x^3 + 5x^2 + 1
Difference: x^3 + x^2 + 9
Product: 3x^5 + 6x^4 + 5x^3 - 2x^2 - 20

Enter a value to evaluate the first polynomial: 2
Value of the first polynomial at 2: 17

C:\Users\ping9\OneDrive\文件\.aino\虎科\2-1_資料結構\Homework_3\src>
```

- **驗證計算(Proving)**

$(3x^2 + 5) + (x^3 + 2x^2 - 4) = x^3 + 5x^2 + 1$。

$(3x^2 + 5) - (x^3 + 2x^2 - 4) = x^3 + x^2 + 9$。

$(3x^2 + 5) * (x^3 + 2x^2 - 4) = (3x^5 + 6x^4 - 12x^2) + (5x^3 + 10x^2 - 20) = 3x^5 + 6x^4 + 5x^3 - 2x^2 - 20$。

$x = 2$，$3x^2 + 5 = 12 + 5 = 17$。

- **效能量測(Measuring)**

```
Compilation results...
--------
- Errors: 0
- Warnings: 3
- Output Filename: C:\Users\ping9\OneDrive\文件\.aino\虎科\2-1_資料結構\Homework_3\src\Hw3.exe
- Output Size: 1.83928394317627 MiB
- Compilation Time: 1.39s
```

● 申論及開發報告

1. 結構 Node
    - ✧ 說明：用來存儲多項式的項目，包含係數(coef)、指數(exp)和指標(link)。
2. 加法(operator+)
    - ✧ 功能：將二個多項式相加並回傳。
    - ✧ 實作：
        1. 使用兩個指標遍歷二個多項式。
            - ♦ 指數不同時，將較大者的項加入結果。
            - ♦ 指數相同時，將兩項的係數相加後加入結果。
        2. 回傳結果。
    - ✧ 說明：使用兩個指針遍歷兩個多項式，按指數大小進行操作。
3. 減法(operator -)
    - ✧ 功能：將二個多項式相減並回傳。
    - ✧ 實作：
        1. 使用兩個指標遍歷二個多項式。
            - ♦ 指數不同時，將較大者的項加入結果。
            - ♦ 指數相同時，將兩項的係數相加後加入結果。
        2. 回傳結果。
    - ✧ 說明：使用兩個指針遍歷兩個多項式，按指數大小進行操作。
4. 乘法(operator*)
    - ✧ 功能：將二個多項式相乘並回傳。
    - ✧ 實作：
        1. 使用兩個指標遍歷二個多項式。
        2. 回傳結果。
    - ✧ 說明：使用兩個指針遍歷兩個多項式，將所有的項相乘。
5. 多項式求值(Evaluate)
    - ✧ 功能：計算多項式於某點的值。
    - ✧ 實作：
        1. 使用指針遍歷多項式。
            - ♦ 計算每項的值：係數 x 值^指數(coef · x^exp)。
        2. 加總並回傳結果。
    - ✧ 說明：可提供直接的計算功能，應用於數值分析或求解。